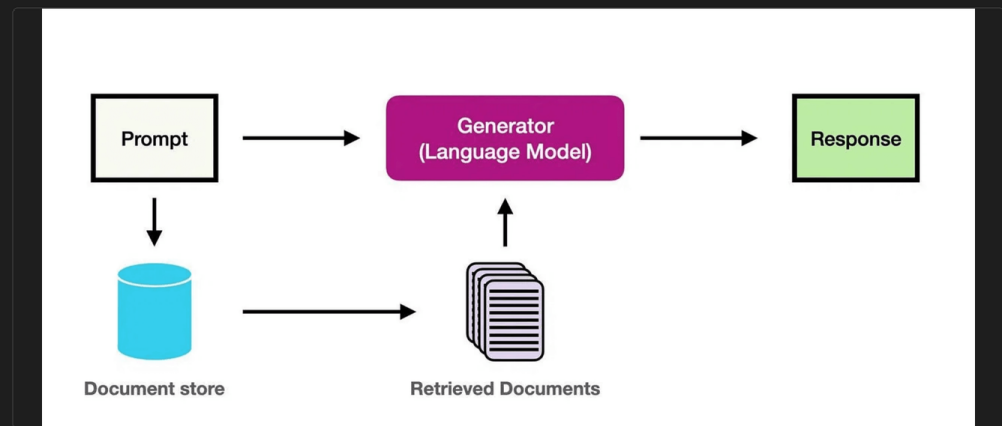


RAG: Retrieval-Augmented Generation

Build it. Break it. Fix it.



The Core Problem: Why LLMs Need RAG

1. Frozen in Time

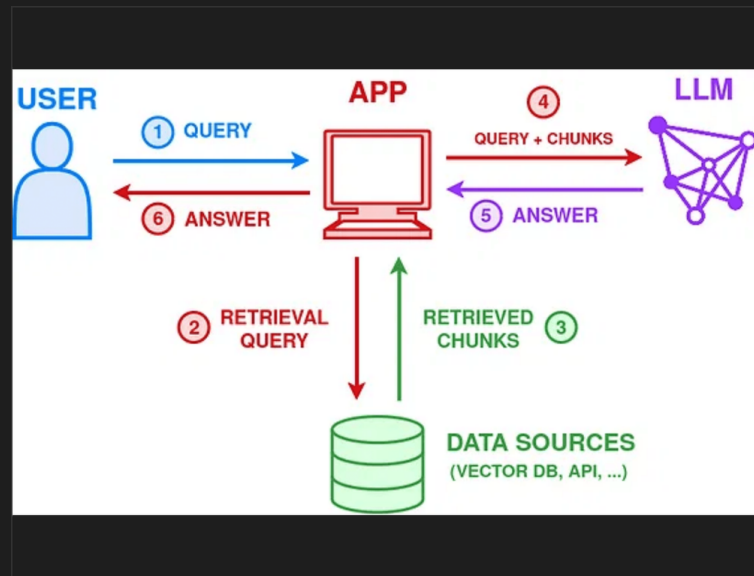
LLMs only know what was in their training data. They lack awareness of current events or recent updates.

2. No Private Knowledge

They haven't seen your **company handbook**, **Slack messages**, or **internal docs**.

3. Confident Hallucination

When they don't know, they confidently invent answers. "It only knows training data. Your docs don't exist in its world."

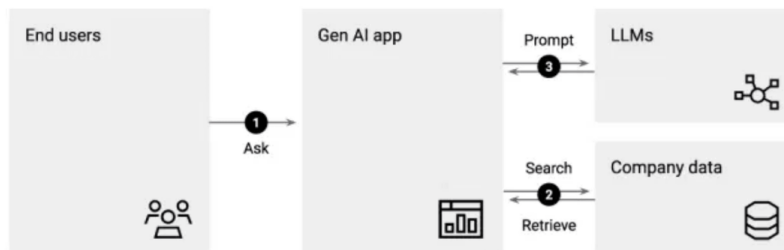


The RAG Workflow and When to Use It

User Question → RETRIEVE → AUGMENT prompt → GENERATE answer

Strategy	When to Use
Context Window	Small knowledge base (< 500 pages)? Stuff it all in.
Fine-tuning	Model needs a new skill, style, or specific behavior.
RAG	Model doesn't KNOW something (external/private data).

Retrieval Augmented Generation (RAG) for AI Applications



Ingestion Part 1: Document Loading and Parsing

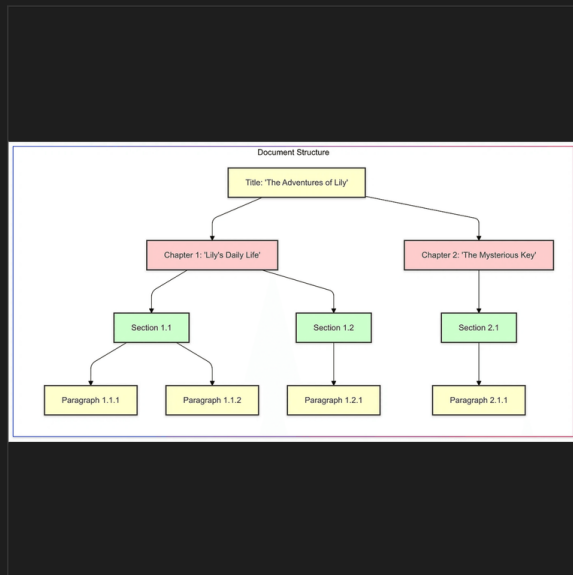
GARBAGE IN, GARBAGE OUT.

Raw documents are rarely clean text:

- PDFs → Tables often destroyed or misaligned
- Word → Structural headers and metadata lost
- HTML → Navigation bars and footers included

Recommended Tools	Best For
Unstructured.io	General purpose, multi-format
LlamaParse	Complex PDFs and table extraction
Docling	High-speed, structure-aware parsing

"If your parser can't handle structure, your RAG is broken before it starts."



Ingestion Part 2: Chunking Strategies

Fixed-Size Chunking

Naive approach: split every N tokens with overlap. Often cuts mid-sentence.

Faithfulness: ~0.47

Recursive Chunking

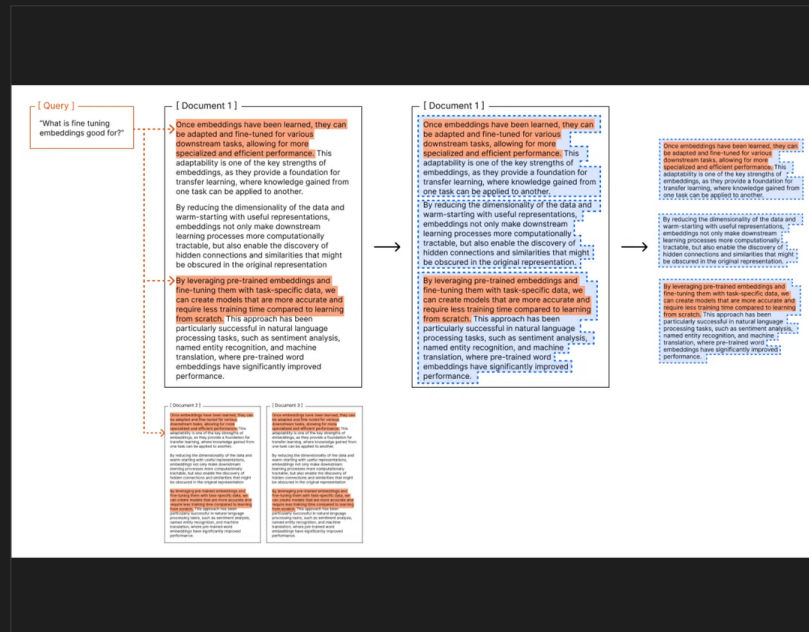
Splits on paragraphs → sentences → characters. Respects natural boundaries.

Faithfulness: ~0.80

Advanced Methods

Structure-aware (headers), AST-based (code), and Parent-Child (retrieve large, embed small).

"This is where 80% of RAG systems fail. Every cut destroys context."



Contextual Retrieval: The Anthropic Approach

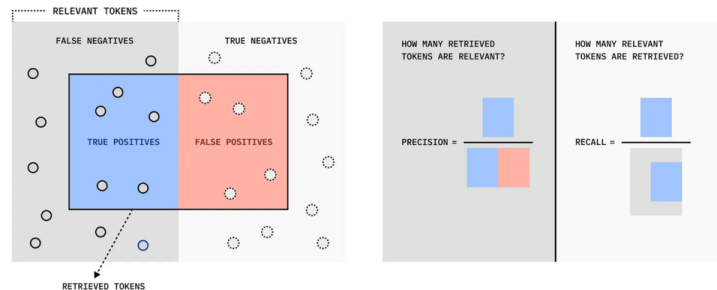
The Problem: Lost Context

A single chunk like "revenue grew 3%" loses context (which company? which year?).

```
// BEFORE:  
"The company's revenue grew by 3%..."  
// AFTER:  
"[ACME Corp SEC filing, Q2 2023. Previous quarter: $314M.]  
The company's revenue grew by 3%..."
```

67% Fewer Failures

When combined with BM25 and reranking. One-time ingestion cost, massive retrieval gain.

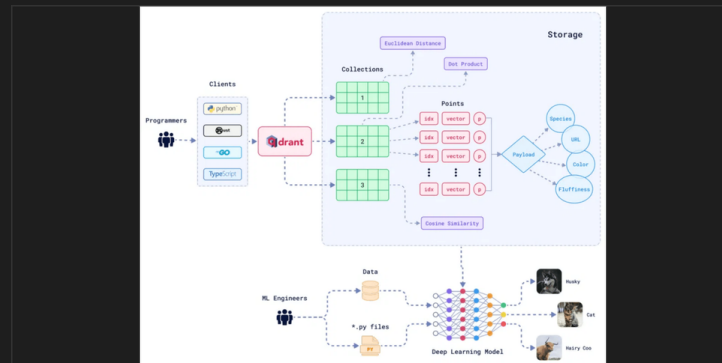
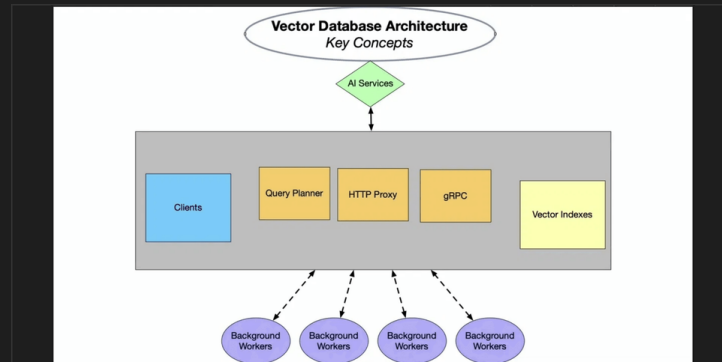


Turning text into high-dimensional vectors. Similar meanings = nearby points in space. Models: **OpenAI**, **Cohere**, **Jina**.

Store millions of embeddings and find top-k matches in milliseconds using [ANN](#) (Approximate Nearest Neighbor).

- **Prototyping:** Chroma, FAISS
- **Production:** Pinecone, Weaviate, Qdrant
- **SQL-Native:** pgvector (Postgres)

Always store **source**, **page**, and **date** alongside vectors for filtering and citations.



Retrieval Part 1: Hybrid Search & Filtering

Hybrid Search: The Best of Both Worlds

Semantic Search (Dense)

Finds **meaning** and concepts. Matches "car" with "automobile" or "vehicle".
Great for natural language queries.

Keyword Search (BM25/Sparse)

Finds **exact terms**, names, and IDs. Essential for queries like "JIRA-2847" or specific product codes.

Combine results using **Reciprocal Rank Fusion (RRF)**. Industry consensus: Hybrid beats semantic alone every time.

Metadata Filtering: The "WHERE" Clause

```
// Filter by date, source, or category BEFORE vector search
query = "Revenue docs FROM 2024"
results = vector_db.search(query, filter={"year": 2024})
```

"If you know the user is asking about Q3, why search Q1 and Q2? Filter first, then search."

Retrieval Part 2: Reranking and Compression

The Two-Pass Retrieval System

First Pass: Bi-Encoder

Fast & Rough. Compresses query and docs separately. Best for scanning millions of records to find the top 50-100 candidates.

Second Pass: Cross-Encoder

Slow & Precise. Sees query and doc together. Understands nuance and re-orders the top candidates to the final top 5.

Contextual Compression

Retrieved chunks are often **90% noise and 10% gold**. Compression uses an LLM to extract only the relevant parts before the final prompt.

"The highest-ROI upgrade after chunking. More important as your data grows."

Query Transformation: Making Questions Better

1. Rewriting

Transforming a **messy user question** into a clean, standalone query optimized for vector search.

2. Multi-Query

Generate **3-5 different phrasings** of the same question. Search all and deduplicate results for higher recall.

3. HyDE (Hypothetical Answer)

Generate a **fake answer** first. Embed that answer instead of the question to find documents with similar content.

4. Step-Back Prompting

Ask a **broader, background question** first to retrieve high-level context before answering the specific query.

"The hypothetical answer is often closer to matching documents than the original question."

RAG Prompt Design and The Full Pipeline

Good Prompt Rules

1. Grounding: "Answer ONLY from provided context."
2. Escape Hatch: "If you don't know, say so."
3. Citations: "Cite your sources with metadata."
4. Structure: Label each chunk clearly.

The End-to-End Pipeline

INGESTION: Docs → Parse → Chunk → Contextualize → Embed → Store



RETRIEVAL: Question → Transform → Hybrid Search → Rerank → Compress



GENERATION: Prompt → LLM → Answer with Citations

"RAG without citations is just a fancier hallucination machine."

The Upgrade Path and Implementation

THE UPGRADE PATH

Level 0: Context Window (Small docs)

Level 1: Naive Semantic Search

Level 2: Better Chunking (Recursive)

Level 3: Contextual Retrieval

Level 4: Hybrid Search (Semantic + BM25)

Level 5: Reranking

Level 6: Query Transformation

HOMEWORK

BUILD: Full RAG pipeline over YOUR docs. Try two chunking strategies and compare results.

READ: anthropic.com/news/contextual-retrieval

THINK: Where did your pipeline fail? What specific upgrade would fix it?